UMIRHACK 2026 · КЕЙС #02

Команда HTTP 418

MADRIGAL PROTOCOL LAB

ЗАДАЧА КЕЙСА

Восстановление недокументированного бинарного протокола поверх TCP

СТЕК ТЕХНОЛОГИЙ

Python 3.12 · PySide6 · dpkt · pytest · Docker

КОМАНДА

Участники разработки



Сабадаш Павел

@Pasha1778

АРХИТЕКТУРА ЗАХВАТ ИНТЕРФЕЙС



Предков Никита

@petruan

ПРАВИЛА ТЕСТЫ РЕВЬЮ

Компактная команда из 2 человек с четким разделением зон ответственности

ПРОБЛЕМА TCP

TCP разрезает сообщения посередине

Границы пакета и границы логического сообщения — разные вещи



Ключевая сложность: Поток TCP не сохраняет границы сообщений. Необработанный сегмент может содержать как часть одного сообщения, так и сразу несколько полных команд.

МЕТОДОЛОГИЯ

Дано и Требуется

ДАНО

- Захваты трафика pcap / pcapng
- Журнал внешних действий устройства
- **Спецификация протокола отсутствует**

ТРЕБУЕТСЯ

- Проверяемая структура сообщений
- Точная локализация ошибок чтения
- Автоматическое выявление версий v1/v2

Уточнение гипотезы смещения (Length Offset)

Смещение длины зафиксировано на offset 2 (2 байта)



АРХИТЕКТУРА СИСТЕМЫ

Модули связаны контрактами JSON

Слабая связность: порядок вызовов не имеет значения

capture

`pcap / pcapng`

Сборка TCP-сессий, обработка provenance

`JSON Schema 1`

reassembly

`protocol + hypothesis`

Правила фрейминга, поиск контрпримеров

`JSON Schema 2`

ui

`PySide6 Desktop`

Интерактивное исследование и просмотр

`JSON Schema 3`

Полный декуплинг: каждый модуль тестируется и запускается независимо

ПРОСЛЕЖИВАЕМОСТЬ

Связь интерпретации с исходными байтами

OFFSET / HEX DUMP PACKET METADATA

```
00000010 00 00 01 10 01 00 00 01 11 01 00 00 01 12 01 00
00000020 00 01 10 01 00 00 01 11 01 00 00 01 12 01 00 00
00000030 01 10 01 00 00 01 11 01 00 00 01 12 01 00 00 01
00000040 10 01 00 00 01 11 01 00 00 01 12 01 00 00 01 10
```

Пакет: #3 Seq: 1001 Timestamp: 1700000000.004 Сессия: s1

Обратная навигация: Любое распаршенное поле кликабельно и подсвечивает точный диапазон байтов в pcap.

АНАЛИЗ АНОМАЛИЙ

Контрпример важнее успешной метрики

⚠ 80 КОНТРПРИМЕРОВ В v1 (Сессии s2, s3)

```
offset 0: 02000003130055 → value 2 not in [1]
offset 7: 020000031400ff → value 2 not in [1]
offset 14: 0200000315009f → value 2 not in [1]
offset 21: 0200000313005c → value 2 not in [1]
offset 28: 02000003140003 → value 2 not in [1]
```

Команда 0x02

Появление байта 0x02 привело к 80 сбоям в первоначальной правиле v1.

Связь с логами

Сопоставление с журналом подтвердило: байт 0x02 соответствует действию «Изменить настройку».

РЕЗУЛЬТАТЫ

Прогресс точности разбора (v1 → v2)

ВЕРСИЯ V1

0.43

60 / 140 сообщений

80 контрпримеров

ВЕРСИЯ V2

1.00

140 / 140 matched

0 контрпримеров

Проверено на реальном датасете организаторов: 45 запросов + 45 ответов (Precision 1.00)

СБОРКА ФРАГМЕНТОВ

Обработка IPv4 и IPv6 фрагментации

IPV4 PROTOCOL

- Группировка по `id` и `protocol`
- Смещение (offset) в блоках по 8 байт
- Неполные фрагменты не собираются

IPV6 PROTOCOL

- Группировка по уникальному `id`
- Контроль перекрытия фрагментов
- Перекрывающиеся датаграммы отбрасываются

12 интеграционных тестов фрагментации

Статус `incomplete` сохраняется до полного покрытия датаграммы



ПРОИЗВОДИТЕЛЬНОСТЬ

Набор организаторов разобран целиком

12 сек

Общее время обработки

350 МиБ

Пик памяти (9% от 4 ГБ)

250k

Обработанных пакетов

Условия замера: 95.66 МиБ pcap, 250 000 пакетов, 1000 сессий, 4 vCPU / 16 ГБ RAM

ГРАНИЦЫ ПРИМЕНИМОСТИ

Честное определение ограничений

ЧТО МЫ НЕ УТВЕРЖДАЕМ

- Смысл байта флагов не установлен (постоянен в дампе)
- Поле значения остается гипотезой, а не фактом

ЗА РАМКАМИ РЕШЕНИЯ

- Альтернативные чтения берутся из заданного набора
- Интерпретация гарантирована только для проверенных захватов

РЕЛИЗ V1.0.0

Инструмент превращает байты в проверяемое знание

ИСХОДНЫЙ КОД ПРОЕКТА

github.com/Pavel1778/madrigal-protocol-lab

@Pasha1778 @petruan